

Cross Traffic Intersection

Ismail Hossain
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
ismail.hossain@stud.hshl.de

Bodrul Amin Murad
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
bodrul-amin.murad@stud.hshl.de
Abu Sayem
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
abu.sayem@stud.hshl.de

Raihan Uddin
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
raihan.uddin@stud.hshl.de

Abstract—This project outlines the design and simulation of an autonomous traffic management system for a four-way intersection utilizing a centralized scheduling algorithm. Unlike conventional traffic signals that function on fixed cycles, this system dynamically determines traffic flows based on current queue lengths to maximize throughput and minimize unnecessary delays. Four logical traffic queues are established to categorize compatible traffic directions, which include straight movements, right turns, and non-conflicting left turns. A dynamic max-weight scheduling algorithm is executed in VHDL to give priority to the queue with the most significant traffic load, thereby ensuring fairness and enhancing parallelism. The system is represented using a finite state machine with compute, green, and all-red gap phases. Functional verification is conducted with ModelSim to validate accurate timing behavior and state transitions. This methodology facilitates intelligent decision-making and establishes a foundation for hardware-level implementation in actual traffic systems.

I. INTRODUCTION

The Cross Traffic Management project deals with the complexity of efficient routing of autonomous vehicles through a four-way intersection. Conventional traffic control methods lack the adaptability and responsiveness needed to manage real-time vehicle flows for autonomous vehicles. To avert these limitations, our implemented system, here, employs a max dynamic weight scheduling algorithm— dynamically assigning priority to each traffic direction based on real-time queue lengths, vehicle arrival rates, and safety constraints.

A. Core Logic in C:

The foundational behavior of the intersection controller was implemented in C. This stage established the data structures and algorithms needed to model vehicle queues and calculate dynamic weights for each direction.

B. FreeRTOS Integration:

To support real-time execution and concurrent processing of sensor data, queue management, and scheduling decisions, the C implementation was ported to FreeRTOS. This ensured deterministic task scheduling and inter-task communication under realistic load conditions.

C. Uppaal Model:

Using UPPAAL, we formally modeled the system's timing behavior and verified critical properties such as deadlock freedom, bounded waiting time, and correct weight recalculation. This step provided mathematical guarantees of system correctness before hardware realization.

D. VHDL and ModelSim Implementation:

Finally, the scheduling logic was translated into VHDL and simulated in ModelSim. This hardware-oriented implementation demonstrates the feasibility of deploying the controller on FPGA or ASIC platforms for ultra-low-latency intersection management.

By combining software development, real-time operating systems, formal verification, and digital circuit design, this project delivers a scalable and robust traffic control solution suitable for smart intersections in autonomous vehicle networks.

II. SysML DIAGRAMS

SysML (Systems Modeling Language) diagrams provide a visual representation of the system's structure and behavior. They help in understanding the flow of information and control within the traffic control system. In this project, we use three key SysML diagrams: Use Case Diagram, Activity Diagram, and requirement diagram.

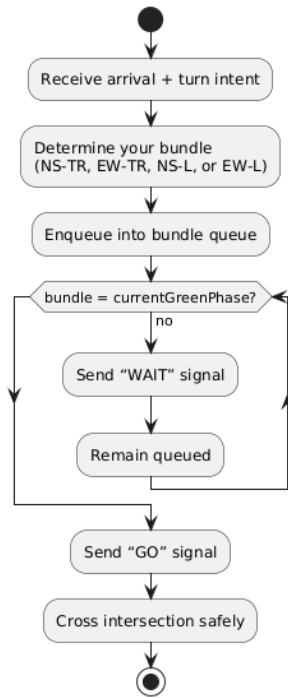


Fig. 1: Activity Diagram

- 1) Receive arrival + turn intent
 - (a) A vehicle arrives at the stop line and communicates its intended movement (straight, left, or right).
- 2) Determine bundle
 - (a) Based on direction (North South or East West) and turn intent (through or left), the vehicle is assigned to one of four "bundles": NS-TR, EW-TR, NS-L, or EW-L.
- 3) Enqueue into bundle queue
 - (a) The controller places the request in the corresponding queue for later scheduling.
- 4) Check against current green phase
 - (a) If the vehicle bundle matches the currently active (green) phase, it bypasses waiting; otherwise:
 - (a) Send "WAIT" signal
 - (b) Remain queued until the green phase switches back.
- 5) Send "GO" signal
 - (a) When its bundle is selected by the scheduling algorithm, the controller grants green.
- 6) Cross intersection safely
 - (a) The vehicle proceeds through the junction under the granted green signal.

A. Requirement Diagram

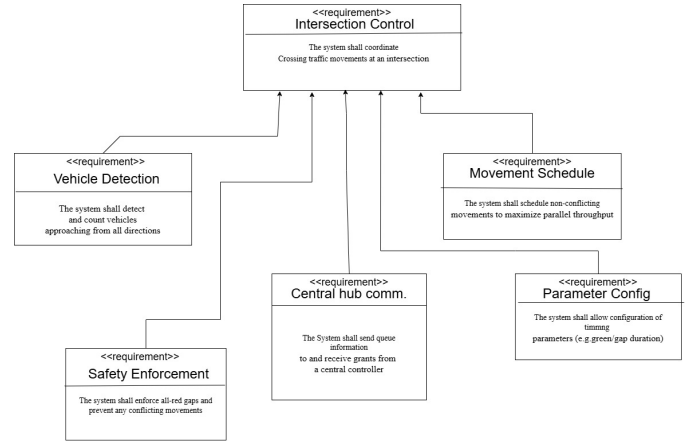


Fig. 2: Requirement Diagram

This UML requirements diagram organizes and decomposes the system-level requirement "Intersection Control" into five constituent requirements:

- 1) Vehicle Detection
 - (a) "The system shall detect and count vehicles approaching from all directions".
- 2) Movement Schedule
 - (a) "The system shall schedule non-conflicting movements to maximize parallel throughput".
- 3) Central Hub Communication
 - (a) "The system shall send queue information to and receive grants from a central controller".
- 4) Parameter Configuration
 - (a) "The system shall allow configuration of timing parameters (e.g., green/gap duration)".
- 5) Safety Enforcement
 - (a) "The system shall enforce all-red gaps and prevent any conflicting movements".

B. Use Case Diagram

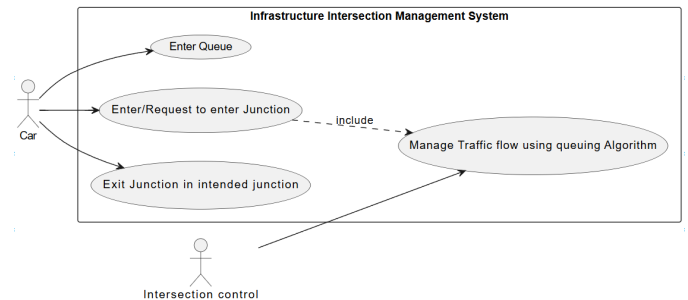


Fig. 3: Requirement Diagram

This diagram captures the key interactions between two actors and the Infrastructure Intersection Management System:

- 1) **Actor:** Car

- (a) Enter Queue
 - (b) Enter/Request to Enter Junction
 - (a) Includes the “Manage Traffic Flow using Queuing Algorithm” use case.
 - (c) Exit Junction
- 2) **Actor:** Intersection Control
- (a) Invokes “Manage Traffic Flow using Queuing Algorithm” (representing the central controller or coordinating entity that runs the scheduling logic).

III. SCHEDULING ALGORITHM

To efficiently manage vehicle movement at a four-way autonomous intersection, we have implemented a Dynamic Max-Weight Scheduling Algorithm as the core of our traffic control logic. The system uses real-time queue lengths to determine the optimal movement pattern that ensures fairness and maximizes throughput.

Queue Structure and Movement Mapping We divided the entire traffic flow into four logical queues based on non-conflicting grouped directions:

- Q_0 : All North-South straight and right-turn movements
- Q_1 : All East-West straight and right-turn movements
- Q_2 : Left-turns from North to West and South to East
- Q_3 : Left-turns from East to North and West to South

This arrangement guarantees that vehicles lined up together can travel concurrently without the possibility of accidents, while facilitating the effective scheduling of various directions at the same time.

Working Principle of the Algorithm At each scheduling cycle, the central controller performs the following operations:

- Arrival Monitoring: Each queue input (Q_0 – Q_3) is continuously updated based on simulated vehicle arrivals.
- Queue Evaluation: The controller compares the size (weight) of each queue.
- Selection: The queue with the highest number of vehicles (i.e., the maximum weight) is selected for the next green phase.
- Execution: The selected queue receives a green signal for a fixed time window (G ticks), allowing vehicles in that group to proceed.
- Gap Phase: After the green period, an all-red interval (Y ticks) is introduced as a safety buffer before re-evaluating the queues.
- Repetition: The process then returns to step 2.

This cyclic selection guarantees dynamic responsiveness to traffic load and avoids starvation of less congested directions over time.

Visual Representation of the Non-conflicting Movements(Queues)

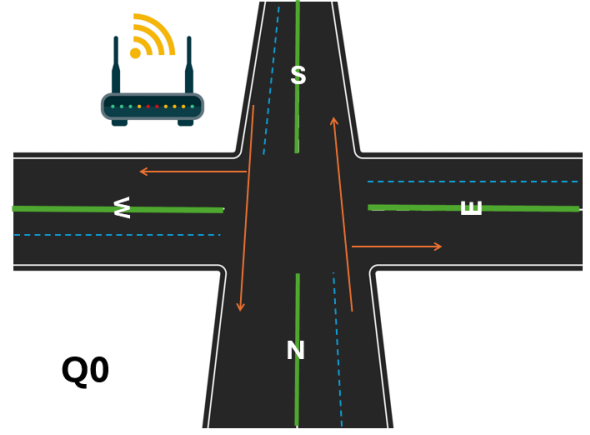


Fig. 4: Visual representation of Q_0 movements.

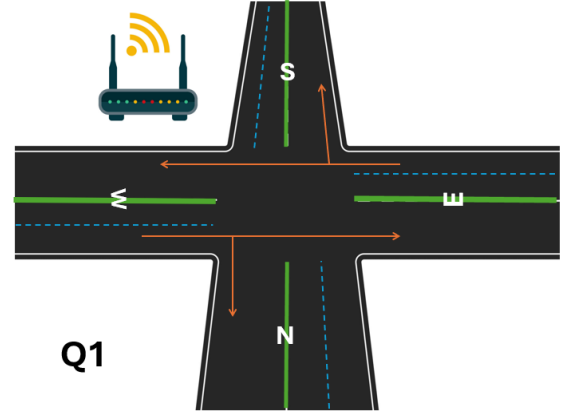


Fig. 5: Visual representation of Q_1 movements

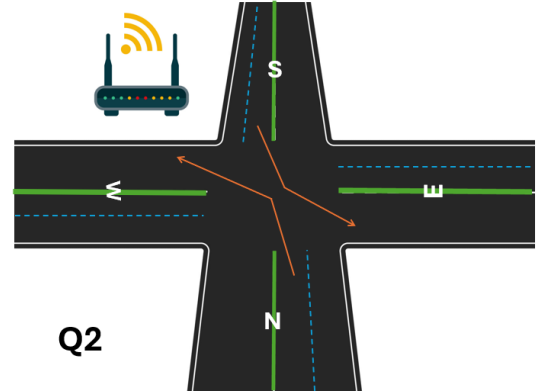


Fig. 6: Visual representation of Q_2 movements

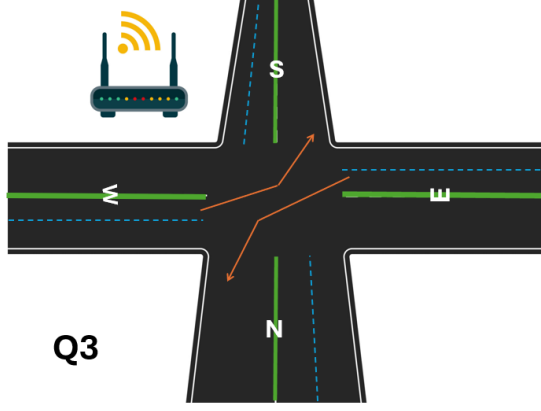


Fig. 7: Visual representation of Q3 movements

IV. UPPAAL MODEL

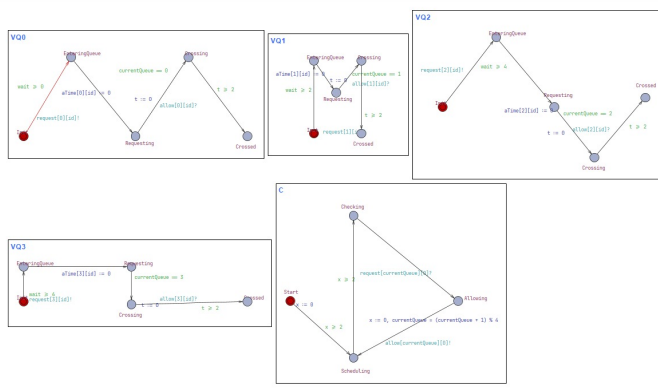


Fig. 8: Uppaal

This UPPAAL model specifies vehicle processes in four templates: TruckQueue0, TruckQueue1, TruckQueue2, and TruckQueue3. Each template describes an independent vehicle queue arriving at an intersection controlled by a traffic signal or controller from different directions or lanes of travel. We use parameterized timed automata templates for each vehicle - where each instance models the actions of a single vehicle under behavioral control of a Controller. Each vehicle has five states: Init, EnteringQueue, Requesting, Crossing, and Crossed. The Init state is the starting state for every vehicle, where the vehicle will stay until it satisfies a minimum wait time according to its queue type to represent staggered arrivals commonly seen in real-world traffic systems. Wait timing is expressed using local timing and guard conditions. TruckQueue0 has no wait time ($\text{wait } \zeta = 0$), TruckQueue1 has at least 2 time units, TruckQueue2 has 4 time units, and TruckQueue3 has 6 time units. After the wait has been satisfied, the vehicle will send a synchronization signal $\text{request}[q][id]!$ indicating its queue number, q , and the vehicle identifier, id , and will enter the next state, EnteringQueue. The vehicle will go to the Requesting state, where it will reset its own arrival timer

($\text{aTime}[q][id] := 0$), then wait for the controller's permission to cross. The permission is controlled by a central Controller process which evaluates all active queues and does select one queue to give access based on current traffic demand. The Controller uses a Max Dynamic Weight Scheduling algorithm in which it computes the weight of each movement phase by summing the queue lengths of the vehicles in motion of that phase. The controller selects the phase with the highest weight (i.e., the most congested phase it selects for access). The controller lets the vehicle queues know they have access by sending the signal $\text{allow}[q][id]!$ to the appropriate vehicle queue(s). If the Requesting state vehicle detects the controller has activated its queue ($\text{currentQueue} == q$) and receives the $\text{allow}[q][id]?$ signal, it transitions to Crossing state. In Crossing state, the vehicle will simulate traversing the intersection and remain for the minimum time of at least 2 time units ($t \zeta = 2$). Once the 2 time units have passed, the vehicle will transition to the last state, Crossed, which signifies the vehicle is now leaving the intersection. To work properly, the vehicle queues will need to be synchronized with the controller. If the vehicle in the began selected by the controller and it is in the queue selected by the controller is not yet in the requesting state and thus not sends the $\text{request}[q][id]!$ signal, the controller will hang waiting for the request it will never receive, leading to a deadlock — confirmed by the failure of the verification query $A[]$ not deadlock. This situation arises because while the controller is dynamically responding to current queue lengths, the model assumes the vehicle will be ready to request access when its queue is selected. The staggered wait times break this assumption, which can delay the system indefinitely. Overall, the TruckQueueX templates accurately describe timed behavior in traffic lane use with synchronization, access control, and fairness based on dynamic load. However, the model also illustrates the need to time vehicle requests relative to the decision cycle of the controller. If reliability and realism are improved, the model could be extended to support dynamic arrivals of vehicles, buffered queues, and non-blocking broadcast for synchronization to increase fault tolerance and reduce deadlock.

Uppaal Sequence

The sequence diagram, we can see the motion of the four vehicle queues (VQ0, VQ1, VQ2, and VQ3) that interact dynamically with the controller (C) when working together to coordinate access to an intersection. Each vehicle queue initiates in the Init state. Once in the Init state, VQ0 is the next vehicle queue to move past the Init state and synchronously send a request signal $\text{request}[0][id]!$ to the controller, indicating that it is ready to occupy the intersection. Upon receipt of the request, C transitions to the Scheduling and Checking sequence with its request from VQ0. The remaining requesters, VQ2 ($\text{request}[1][id]!$), and VQ3 ($\text{request}[2][id]!$), remain in an Init state and send its respective request signals after its configurable delay times and moves to the requesters and waits for permission to occupy the intersection. When all available requests have been sent, the controller evaluates

which queue is next to receive access to the intersection, based on the max dynamic weight scheduling algorithm. For every traffic phase, C calculates weights based on the current queue lengths and identifies which phase has the highest demand. C then proceeds and enters its Allowing state and sends out an `allow[currentQueue][0]!` signal to the next queue in line to cross. In this sequence, it corresponds to VQ0, which then transitions into the Crossing state representing occupancy of the intersection. C, having provided intersection access, begins looping back to evaluate queue weights and schedule in real-time. The sequence diagram represents the intended and expected behavior of the system. Vehicles are permitted access when they are ready and the controller provides permission based on queue priority.

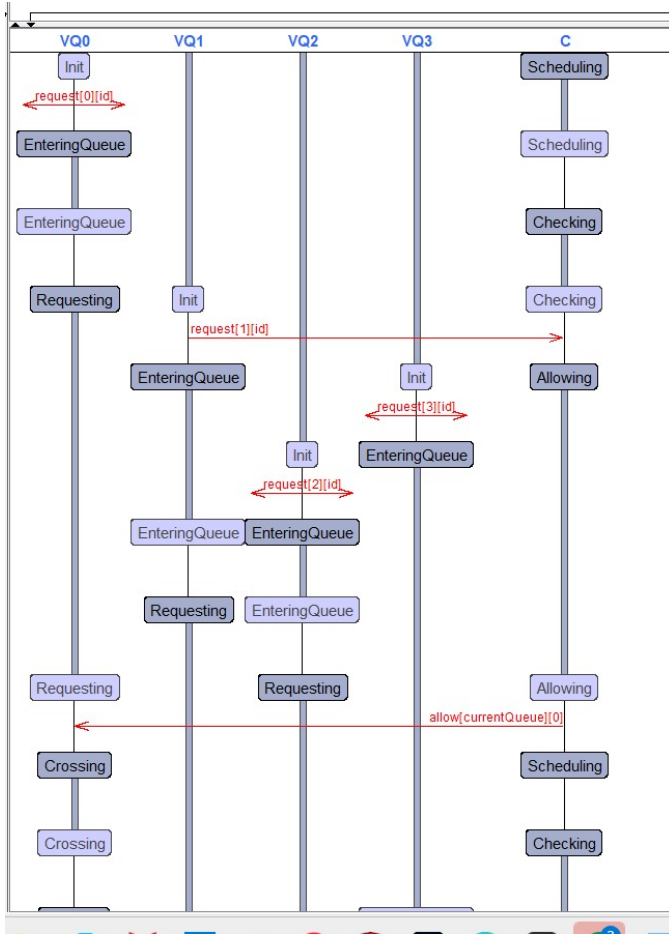


Fig. 9: Uppaal Sequence

V. C AND FREERTOS IMPLEMENTATION

For the C and FreeRTOS implementation we have used ESP32 board since it has in built FreeRTOS capability. We have presented the coding and the outline below:

```
#include <Arduino.h>
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include <freertos/semphr.h>

// - PARAMETERS (in milliseconds) -
static const uint32_t MIN_ARRIVAL = 5000; // 5-10 s between arrivals
static const uint32_t MAX_ARRIVAL = 10000;
static const uint32_t GREEN_TIME = 8000; // 8 s green
static const uint32_t GAP_TIME = 2000; // 2 s all-red

// - QUEUE LENGTHS & LABELS -
static int q[4] = {0, 0, 0, 0};
static const char* phaseName[4] = {
    "NS Through/Right",
    "EW Through/Right",
    "NS Left",
    "EW Left"
};
static const char* arrNames[4] = {
    "Arrival0", "Arrival1", "Arrival2", "Arrival3"
};

// - MUTEX for protecting q[] increments/reads -
SemaphoreHandle_t qMutex;
```

Fig. 10: Library and Variable Declaration

In the first part of the code, we include all the libraries required for this project. These include the standard `Arduino.h` library and FreeRTOS libraries such as `task.h` for creating tasks and `semphr.h` for using semaphores. FreeRTOS enables concurrent execution of multiple tasks, which is crucial in this project because traffic from different directions must be handled independently.

Next, we define some important parameters that govern the behavior of the simulation. For example, `MIN_ARRIVAL` and `MAX_ARRIVAL` set the range for how frequently cars arrive — between 5 to 10 seconds. The `GREEN_TIME` parameter determines how long the green light remains active for a selected lane (8 seconds), while `GAP_TIME` introduces a short all-red safety interval (2 seconds) between signal transitions.

We also initialize an integer array `q[4]` to track the number of cars waiting in each of the four directions. Each index corresponds to a specific traffic direction, such as “NS Through/Right” or “EW Left.” To facilitate logging and debugging, two additional arrays are declared: `phaseName[]` for naming the directions and `arrNames[]` for identifying the arrival tasks.

Furthermore, a semaphore named `qMutex` is declared to safeguard access to the shared `q[]` array. This prevents concurrent access issues by ensuring that only one task modifies the array at any given time. Without this protection, simultaneous updates from multiple tasks could lead to data corruption and unpredictable behavior.

```

// Arrival Task: simulates one bundle's random arrivals
// -----
void arrivalTask(void* pvParameters) {
    int idx = (int)(intptr_t)pvParameters;
    for (;;) {
        // wait a random interval
        vTaskDelay(pdMS_TO_TICKS(random(MIN_ARRIVAL, MAX_ARRIVAL)));

        // safely increment queue length
        xSemaphoreTake(qMutex, portMAX_DELAY);
        q[idx]++;
        Serial.printf("[ARRIVAL] %s → q[%d] = %d\n",
                      phaseName[idx], idx, q[idx]);
        xSemaphoreGive(qMutex);
    }
}

```

Fig. 11: Arrival Task

In this part of the code, we define the `arrivalTask()` function. This function is responsible for simulating cars arriving randomly at one of the four traffic directions. A loop is used to create four separate tasks from this single function—one for each lane or direction.

Inside the function, an infinite loop (`for(;;)`) ensures that cars continue to arrive at each direction indefinitely. Randomness is introduced using the `vTaskDelay()` and `random(MIN_ARRIVAL, MAX_ARRIVAL)` functions, so each lane receives cars at slightly different time intervals, mimicking real-world traffic patterns.

After the delay, the task increments the number of waiting cars in its assigned direction by updating `q[idx]`. This critical section is protected using the semaphore functions `xSemaphoreTake()` and `xSemaphoreGive()`, which lock and unlock the `qMutex` around the update operation. This mechanism prevents race conditions and ensures data consistency by allowing only one task to access the shared `q[]` array at a time.

Additionally, each arrival is logged to the serial monitor using `Serial.printf()`. This message indicates the direction where a car has arrived and displays the updated queue count. Such debug output is valuable during development and testing phases to verify system behavior and logic correctness.

```

// Scheduler Task: picks the heaviest queue & serves it
// -----
void schedulerTask(void*) {
    // give a head start for some arrivals
    vTaskDelay(pdMS_TO_TICKS(2000));

    for (;;) {
        // wait until at least one car is waiting
        int total;
        do {
            vTaskDelay(pdMS_TO_TICKS(500));
            xSemaphoreTake(qMutex, portMAX_DELAY);
            total = q[0] + q[1] + q[2] + q[3];
            xSemaphoreGive(qMutex);
        } while (total == 0);

        // compute & pick the heaviest queue
        xSemaphoreTake(qMutex, portMAX_DELAY);
        int best = 0;
        for (int i = 1; i < 4; ++i) {
            if (q[i] > q[best]) best = i;
        }
        int served = q[best];
        q[best] = 0;
        xSemaphoreGive(qMutex);

        // announce green start
        Serial.printf("\n>> GREEN START: %s\n",
                      "    Cars served: %d\n",
                      "    Queues now:  [%d, %d, %d, %d]\n",
                      phaseName[best], served,
                      q[0], q[1], q[2], q[3]);

        // hold green
        vTaskDelay(pdMS_TO_TICKS(GREEN_TIME));
        Serial.printf("<< GREEN END:  %s\n", phaseName[best]);

        // all-red gap
        Serial.println("*** ALL-RED GAP ***");
        vTaskDelay(pdMS_TO_TICKS(GAP_TIME));
    }
}

```

Fig. 12: Selection of heaviest queue

This part of the code defines the main scheduler, which acts as the central controller of the system. It continuously monitors the queue lengths and decides which direction should receive the green signal next. Like the arrival tasks, it is implemented as a FreeRTOS task and runs independently in parallel.

At the start, the scheduler waits for 2 seconds to allow some cars to accumulate in the queues. Within its infinite loop, it first checks whether any of the four queues contain waiting cars. If all queues are empty, it delays briefly before checking again. Once cars are detected, it compares the queue values and selects the direction with the highest count.

After identifying the busiest lane, the scheduler resets that lane's queue count to zero, announces the green light via a `Serial.printf()` message, and holds the green phase for 8 seconds using `vTaskDelay()`. This simulates vehicles passing through the intersection.

When the green phase ends, it prints a message to indicate the end of the green period and introduces a 2-second all-red safety interval before continuing to the next scheduling cycle.

This scheduler logic effectively demonstrates a max-weight scheduling strategy, where priority is always given to the direction with the highest number of waiting vehicles. Such a strategy helps reduce congestion by dynamically adapting to real-time traffic conditions.

Serial Monitor Output

```

[ARRIVAL] NS Through/Right → q[0] = 5
[ARRIVAL] EW Through/Right → q[1] = 4
[ARRIVAL] NS Left → q[2] = 2
[ARRIVAL] EW Left → q[3] = 1
<< GREEN END:   EW Left
** ALL-RED GAP **

>> GREEN START: NS Through/Right
    Cars served: 5
    Queues now:  [0, 4, 2, 1]
[ARRIVAL] NS Through/Right → q[0] = 1
[ARRIVAL] EW Through/Right → q[1] = 5
[ARRIVAL] NS Left → q[2] = 3
[ARRIVAL] EW Left → q[3] = 2
<< GREEN END:   NS Through/Right
** ALL-RED GAP **
[ARRIVAL] NS Through/Right → q[0] = 2
[ARRIVAL] EW Through/Right → q[1] = 6

>> GREEN START: EW Through/Right
    Cars served: 6

```

Fig. 13: Implementation Output

This is the serial monitor output from the ESP32 simulation, this output shows the real-time behavior of our traffic light scheduler. As we can see, cars are arriving randomly at all four directions, and each arrival updates the corresponding queue. The system continuously checks which queue has the highest number of waiting cars and then gives the green signal to that direction. For example, after serving EW Left, the system identifies that NS Through/Right has 5 cars waiting, so it gives green to that lane and serves all 5 cars. The queue is then reset to 0, while new cars continue arriving at other directions. Later, EW Through/Right becomes the heaviest with 6 cars, so it gets the green light next. We also see the "ALL-RED GAP" messages between green phases, simulating a real-world delay between traffic light switches. This output confirms that our FreeRTOS-based scheduler is functioning correctly, prioritizing the busiest direction and updating queues dynamically.

VI. VHDL AND MODELSIM IMPLEMENTATION

To verify the logic of our dynamic traffic scheduling algorithm in terms of hardware performance, we implemented the system using VHDL (VHSIC Hardware Description Language) and conducted simulations in ModelSim. The VHDL module represents a finite state machine (FSM) that consistently observes four traffic queues and dynamically identifies the most congested queue to activate the corresponding green

signal. The simulation illustrates phase-based scheduling, the duration of the green light, and the safety intervals during the all-red phase. This section details the key components of the VHDL implementation, including the entity declaration, the primary process block, and the logic for controlling outputs.

```

5  entity scheduler is
6      generic (
7          G : integer := 4;
8          Y : integer := 2
9      );
10     port (
11         clk      : in  std_logic;
12         reset_n   : in  std_logic;
13         q0, q1, q2, q3 : in  unsigned(3 downto 0);
14         best      : out unsigned(1 downto 0);
15         green     : out std_logic_vector(3 downto 0);
16         all_red   : out std_logic
17     );
18 end entity scheduler;
19

```

Fig. 14: Entity Declaration

This entity outlines the interface for the traffic scheduler module. It accepts four unsigned 4-bit inputs—q0 to q3—which represent the number of vehicles waiting in each queue. The module also receives a clock input signal, `clk`, and an active-low reset signal, `reset_n`.

The outputs include the index of the selected queue (`b`), a one-hot encoded green signal (`green`) indicating which queue is currently being serviced, and an `all_red` signal that ensures a safety interval between phase transitions.

Two generic parameters, `G` and `Y`, define the durations for the green phase and the all-red interval, respectively. These generics provide configurability, allowing the module to adapt to different traffic timing requirements.

```

28 begin
29   if reset_n = '0' then
30     state <= COMPUTE;
31     cnt <= 0;
32     b <= (others => '0');
33
34   elsif rising_edge(clk) then
35     case state is
36
37       when COMPUTE =>
38         b <= "00";
39         if q1 > q0 then
40           b <= "01";
41         end if;
42         if q2 > q0 and q2 > q1 then
43           b <= "10";
44         end if;
45         if q3 > q0 and q3 > q1 and q3 > q2 then
46           b <= "11";
47         end if;
48         cnt <= 0;
49         state <= GREEN_S;
50
51       when GREEN_S =>
52         if cnt < G-1 then
53           cnt <= cnt + 1;
54         else
55           cnt <= 0;
56           state <= GAP_S;
57         end if;
58
59       when GAP_S =>
60         if cnt < Y-1 then
61           cnt <= cnt + 1;
62         else
63           cnt <= 0;
64           state <= COMPUTE;
65         end if;
66
67     end case;
68   end if;
69 end process;

```

Fig. 15: Main Process Block

This section describes the main process block that implements the sequential logic of the finite state machine (FSM). At each rising edge of the clock, the state machine transitions through different states based on current conditions.

In the `COMPUTE` state, the system compares the values of all four queue inputs and assigns the index of the queue with the highest count to the variable `b`. This determines which direction should receive the green signal next.

The `GREEN_S` state activates the selected green signal by asserting the corresponding bit in the one-hot encoded output. A counter is incremented with each clock cycle to maintain the green signal for a fixed duration specified by the generic `G`.

After the green phase, the FSM enters the `GAP_S` state. This state enforces an all-red period by deactivating all green outputs and maintaining the `all_red` signal for a safety interval defined by the generic `Y`. Once the gap duration is complete, the system transitions back to the `COMPUTE` state, and the process repeats.

A. ModelSim Simulation

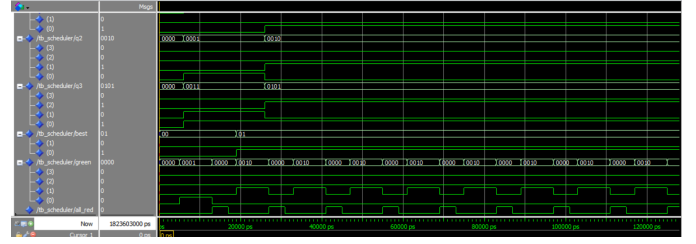


Fig. 16: ModelSim Waveform

To assess the performance of our traffic scheduler, which is based on VHDL, we conducted simulations using ModelSim. The waveform depicted in the figure demonstrates the timing characteristics of the primary signals in the system.

- **Inputs (q0–q3):** These represent four queues corresponding to different traffic directions and movements.
- **Best Signal:** This 2-bit output represents the index of the queue selected by the scheduler as the highest priority based on queue length. As observed in the waveform, the value of best updates during the `COMPUTE` phase, e.g., 01 implies queue 1 is selected as the heaviest.
- **Green Signal:** A 4-bit signal controlling which queue gets the green light. Only one bit is asserted high (1) at a time, corresponding to the selected queue. For instance, if best = "01", then green(1) goes high while others remain low. This is consistent with the waveform.
- **all_red Signal:** This signal indicates a gap phase (all red lights ON) between green phases. It stays high for `Y` clock cycles. In the simulation, it periodically rises between green phases, ensuring no cars move during unsafe switching intervals.

The waveform demonstrates the correct sequential behavior of the system states.

Initially, the system enters the `COMPUTE` state, where it performs queue comparisons to determine the direction with the highest vehicle count. Following the computation, the system transitions into the `GREEN_S` state, where the corresponding green signal is activated for the selected queue. This activation is maintained for a fixed duration of `G` clock cycles.

After the green phase, the FSM moves into the `GAP_S` state, during which the `all_red` signal is asserted (`all_red` = 1). This introduces a short all-red interval for safety before repeating the cycle.

The simulation confirms the correct functioning of the state machine logic, including queue comparison, proper signal toggling, and green-light cycling with appropriate safety gaps between transitions.

VII. CONCLUSION

In this project, we designed and executed a centralized autonomous traffic management system for a four-way intersection, utilizing a queuing-based strategy along with the Dynamic Max-Weight scheduling algorithm. The main goal

was to enhance throughput while reducing wait times and ensuring safe vehicle movements without collisions.

The intersection was conceptualized into four logical queues, each integrating non-conflicting traffic flows (for instance, straight + right, or safe left-turn combinations). Depending on the lengths of the queues, the central controller dynamically chose the queue with the highest weight to receive the green light. To ensure safety, a short all-red interval was included between transitions.

The system was implemented across three domains:

UPPAAL was employed to model and verify the timed automata that governed the traffic flow logic.

C and FreeRTOS on ESP32 facilitated a software simulation where vehicles arrived randomly, queues were managed, and selected phases were displayed on the serial monitor in real time.

VHDL and ModelSim provided a hardware-level realization of the controller, simulating queue selection and green signal output based on the real-time weights of the queues.

The results obtained from both simulation and waveform analysis confirmed the validity of our approach. This project showcases a scalable, modular solution that is suitable for future autonomous traffic settings, effectively balancing efficiency, safety, and fairness in managing intersections.